

CSE 4345: Big Data Analytics

Week 7, Part 1 — Querying Big Data: Hive, HiveQL & the SQL-on-Hadoop Ecosystem

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Course & Lecture Information

Lecture Identity

Course: CSE 4345
Week / Part: 7 of 11 | Part 1 of 2
CLO: CLO3, CLO4
PLO: PLO(e), PLO(f)

CLO3 & CLO4

CLO3: *Explain big data techniques and tools used to store, manage, and analyse large datasets*

CLO4: *Solve big data problems and build a project applying big data tools and frameworks*

Course Progression

Week 5: YARN, file formats (Parquet/ORC/Avro)

Part 1 Covers

- Why SQL-on-Hadoop? The analyst accessibility gap
- SQL-on-Hadoop ecosystem: Hive, Presto, Impala, Drill, Spark SQL
- Hive architecture: HiveServer2, Driver, Metastore, execution engine
- HiveQL query compilation: parse → analyse → optimise → MapReduce/Tez/Spark
- The Hive Metastore: what it stores and why it is the control plane
- Schema-on-read vs. schema-on-write: tradeoffs and when to use each
- HiveQL vs. SQL: key differences (ACID, UPDATE, latency)

Lecture Agenda

The SQL-on-Hadoop Movement

Why SQL-on-Hadoop? The Analyst Accessibility Gap

The Problem Before Hive (pre-2008)

Hadoop's power came with a steep learning curve:

- Every query had to be written as a **Java MapReduce program** — 100+ lines of boilerplate for a simple GROUP BY
- Business analysts, data scientists, and BI teams speak **SQL** — not Java
- Time-to-insight for a simple aggregation: **days** (write, test, debug MapReduce job)
- Facebook needed to enable thousands of non-engineers to query petabytes of user data

The Solution: Declare, Don't Program

Apache Hive (2008, Facebook) introduced a **SQL-like**

Bangladesh Relevance

Government agencies and corporations in Bangladesh generate massive structured datasets but cannot staff armies of Java engineers:

- **NID Database** (~110 million records): DGHS analysts query demographic health correlations using HiveQL over Parquet files on HDFS
- **bKash**: finance analysts run daily revenue reports via HiveQL on 500 M+ transaction records without touching Java
- **a2i (ICTA)**: government e-service logs queried via Hive for policy analytics

Hive democratizes big data analytics beyond the engineering team.

SQL-on-Hadoop Ecosystem Overview

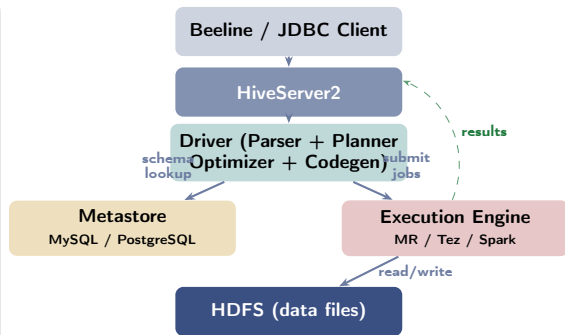
Tool	Year / Origin	Execution Model	Latency	Best For
Apache Hive	2008 Facebook	MR / Tez / Spark (compile-time)	Minutes	Batch analytics large ETL jobs
Presto (Trino)	2013 Meta (Facebook)	In-memory pipeline execution	Seconds	Interactive BI dashboards
Impala	2012 Cloudera	Native C++ MPP engine	Seconds	Low-latency SQL on HDFS/S3
Spark SQL	2014 Databricks	Catalyst optimizer in-memory RDDs	Seconds	Unified batch + streaming SQL
Apache Drill	2012 MapR	Schema-free JSON-native	Seconds	Exploratory SQL on JSON/Parquet
Shared Storage: HDFS S3 Azure ADLS GCS				

Apache Hive Architecture

Hive Architecture: From HiveQL to HDFS

Component Roles

- **HiveServer2**: JDBC/ODBC server; authenticates users; accepts HiveQL queries from Beeline CLI, BI tools (Tableau, Power BI), and JDBC drivers
- **Driver**: core query processing pipeline — Parser → Semantic Analyser → Logical Planner → Query Optimizer → Physical Plan Generator
- **Metastore**: relational database (MySQL / PostgreSQL) storing all metadata: table schemas, column types, partition information, file locations on HDFS
- **Execution Engine**: pluggable — can be **MapReduce** (default legacy), **Tez** (DAG of tasks, faster), or **Spark** (SET hive.execution.engine=spark)
- **HDFS**: actual data storage — Hive tables are



The Hive Metastore: The Control Plane of Your Data Lake

What the Metastore Stores

The Metastore is a **relational database** (typically MySQL or PostgreSQL) that acts as the **central catalogue** for all structured data on HDFS:

- **Database & Table definitions:** column names, data types, comment descriptions
- **Partition metadata:** which partition values exist (e.g., year=2024, month=1) and the HDFS path for each partition directory
- **SerDe (Serializer/Deserializer):** how to parse the physical file format (ORC, Parquet, CSV, JSON) into column values
- **Table statistics:** row counts, column min/max, NULL counts (used by the query optimizer)
- **Privileges:** table-level and column-level access

Why the Metastore is Critical

Without the Metastore, Hive cannot answer a single query — it needs to know *where* the data lives on HDFS and *how* to parse it. This is why the Metastore is often deployed as a **highly available service** (Metastore HA) in production clusters.

Spotify Scale

Spotify's Hive Metastore catalogs over **1 million tables** across a cluster processing 600 billion events per day. The Metastore itself runs on a replicated MySQL cluster with automatic failover. **Lesson:** The Metastore is as important as HDFS itself. Metastore corruption = inability to query any table even though data on HDFS is intact.

Schema-on-Read vs. Schema-on-Write

Schema-on-Read vs. Schema-on-Write

Property	Schema-on-Write (RDBMS)	Schema-on-Read (Hive)
When schema enforced	At INSERT / LOAD time	At SELECT / query time
Invalid data handling	Rejected immediately	Stored; NULL returned at query time
Flexibility	Rigid: must define schema upfront	Flexible: schema applied to any file
Write speed	Slower (validation overhead)	Fast: raw file copy to HDFS
Query speed	Faster (data pre-validated)	Slower (parse & validate per query)
Multiple schemas	One schema per table	Many schemas on same data
Storage format	Proprietary (InnoDB pages)	Open: ORC, Parquet, CSV, JSON, Avro

HiveQL: Syntax, Partitioning, and Bucketing

HiveQL vs. SQL: Key Differences

Feature	SQL (RDBMS)	HiveQL
Schema enforcement	On write (strict)	On read (flexible)
ACID transactions	Full: INSERT, UPDATE, DELETE, ROLLBACK	Limited: ORC tables with ACID mode only
UPDATE / DELETE	Any row, any time	Only in ACID-enabled transactional tables
Indexing	B-tree, hash, bitmap	Partitions + Buckets (no traditional indexes)
Sub-queries	Correlated sub-queries	Non-correlated only (some limitations)
Query latency	Milliseconds	Minutes (batch MapReduce) / Seconds (Tez/Spark)

HiveQL: Table Creation and Basic Queries

Creating and Loading a Table

```
-- External table: data stays in HDFS
-- even if table is dropped
CREATE EXTERNAL TABLE transactions (
  txn_id    BIGINT,
  user_id   STRING,
  amount    DOUBLE,
  country   STRING,
  txn_date  DATE
)
ROW FORMAT DELIMITED
  FIELDS TERMINATED BY ','
STORED AS ORC
LOCATION '/data/bkash/txns/';

-- Load data (moves file to table dir)
LOAD DATA INPATH
  '/staging/txns_2024.orc'
  INTO TABLE transactions;
```

HiveQL Aggregation Queries

```
-- Top 5 countries by total amount
SELECT    country,
          SUM(amount) AS total_BDT
FROM      transactions
WHERE     YEAR(txn_date) = 2024
GROUP BY  country
ORDER BY  total_BDT DESC
LIMIT     5;

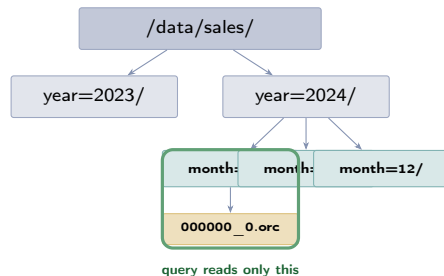
-- Monthly revenue trend
SELECT    MONTH(txn_date) AS mo,
          COUNT(*)        AS num_txns,
          AVG(amount)      AS avg_amount
FROM      transactions
WHERE     country = 'BD'
          AND YEAR(txn_date) = 2024
GROUP BY  MONTH(txn_date)
ORDER BY  mo;
```

Partitioning: Directory-Level Data Pruning

What Is Partitioning?

Partitioning physically organises table data into **separate HDFS sub-directories** based on the values of one or more columns. When a query filters on the partition key, Hive reads **only the matching sub-directories** — skipping all others.

- **Partition key:** typically a low-cardinality column used frequently in `WHERE` (date, country, region, status)
- **Benefit:** avoids full-table scan; reduces data read from HDFS; cuts job time proportionally
- **Example:** query on `year=2024, month=01` reads only 1 sub-directory, not 36 months of data



`WHERE year=2024 AND month=01`

skips year=2023/ and month=02..12/

Partitioning: DDL and Dynamic Partition Insert

Creating a Partitioned Table

```
CREATE TABLE sales (  
    txn_id    BIGINT,  
    amount    DOUBLE,  
    country   STRING  
)  
PARTITIONED BY (year INT, month INT)  
STORED AS ORC;  
  
-- Static partition load  
INSERT INTO sales  
    PARTITION (year=2024, month=1)  
SELECT txn_id, amount, country  
FROM    raw_sales  
WHERE   sale_year = 2024  
        AND sale_month = 1;
```

Dynamic Partition Insert (all months at once)

```
-- Enable dynamic partitioning  
SET hive.exec.dynamic.partition=true;  
SET hive.exec.dynamic.partition.mode  
    =nonstrict;  
  
-- Hive infers partition values  
-- from the last SELECT columns  
INSERT INTO sales  
    PARTITION (year, month)  
SELECT txn_id,  
        amount,  
        country,  
        YEAR(sale_date) AS year,  
        MONTH(sale_date) AS month  
FROM    raw_sales;
```

Query Pruning in Action

Rule

Bucketing: Hash-Based Clustering for Efficient JOINS

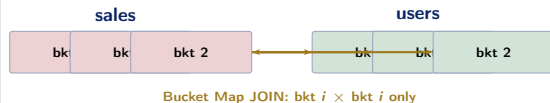
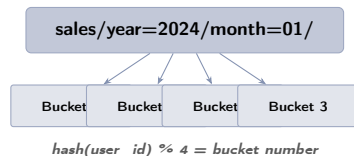
What Is Bucketing?

Bucketing further divides each partition into a **fixed number of files** (buckets) using a **hash function** on a chosen column. Rows with the same $\text{hash}(\text{column})$ land in the same bucket file.

- 1 Define `CLUSTERED BY (column) INTO N BUCKETS`
- 2 All rows where $\text{hash}(\text{column}) \% N = k$ are written to bucket file k
- 3 Result: same bucket file across two tables contains rows that **will join together**

Bucket Map Join (the main benefit):

- Both tables bucketed on join key with the **same number of buckets**



Bucketing DDL

```
CLUSTERED BY (user_id) INTO 32 BUCKETS STORED AS  
ORC;
```

Presto: Interactive SQL on Hadoop

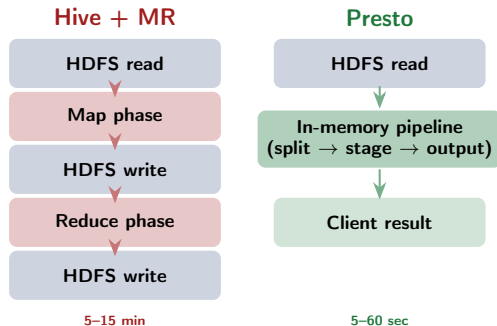
Presto: In-Memory SQL, 10× Faster than Hive+MapReduce

Why Presto Was Built (Meta, 2013)

Facebook analysts needed **interactive** (sub-second to second) SQL queries over petabytes of data on HDFS. Hive+MapReduce could not deliver this — even a simple filtered COUNT took minutes. **Presto**

key design decisions:

- ❶ **No intermediate disk writes:** data flows as a pipeline through the cluster entirely in memory
- ❷ **Vectorized execution:** processes column chunks (batches of 1,024 rows) rather than row by row
- ❸ **Cost-based optimizer:** uses table statistics to choose optimal join order and join strategy
- ❹ **Connector architecture:** same SQL engine for HDFS, Hive, Cassandra, MySQL, Kafka, and S3 simultaneously in one query



Ivy League Alignment

Hive & SQL-on-Hadoop in Top University Curricula

Stanford CS246 — Mining of Massive Datasets

Stanford frames Hive as the canonical example of **declarative MapReduce** — the same principle as relational algebra on distributed data:

- HiveQL's GROUP BY + aggregation compiles to a MapReduce job where Map emits (key, value) pairs and Reduce applies the aggregate function — *this is the Relational Algebra γ operator distributed*
- CS246 assignment: **write a matrix multiply in HiveQL** and compare to raw MapReduce implementation — motivating why SQL abstractions matter even for non-relational computations
- **Exam question type:** “Given a HiveQL query with two JOINS and a GROUP BY, draw the MapReduce DAG that Hive would generate, identifying the shuffle key at each stage.”

MIT 6.830 — Database Systems

MIT covers Hive as a **bridge between traditional RDBMS and distributed systems**:

- Schema-on-read vs. schema-on-write is a core concept in MIT's coverage of data lakes
- The Hive Metastore is compared to PostgreSQL's pg_catalog — both are the system catalogue, but Hive's operates over HDFS
- MIT students implement a simplified Hive planner that converts SQL parse trees into MapReduce job chains

CMU 15-721 — Advanced DB Systems

Student Assessment Pool

Instructions

Select the single best answer. Correct answers **bolded** for instructor reference.

Q1. In Hive, where is table schema metadata (column names, types, partition info) stored?

- ☐ a HDFS NameNode
- ☒ b **Hive Metastore (relational DB such as MySQL)**
- ☐ c HBase
- ☐ d ZooKeeper

Q2. What does “schema-on-read” mean in the context of Hive?

- ☐ a Schema is validated when data is inserted into the table
- ☒ b **Schema is applied when data is queried, not when it is stored on HDFS**
- ☐ c Schema is stored separately from data in HDFS metadata
- ☐ d Schema is inferred automatically from column names in the file header

Instructor Notes

Q1: A common distractor is the NameNode — it stores HDFS block metadata, not table schemas. **Q2:**

- Q3.** Which Hive optimization technique physically organises data into separate HDFS sub-directories by a column's value to avoid full table scans?
- ☐ (a) Bucketing
 - ☐ (b) Indexing
 - ☒ (c) **Partitioning**
 - ☐ (d) Compression
- Q4.** Presto's primary performance advantage over Hive+MapReduce for interactive queries is:
- ☐ (a) Presto supports more SQL functions than HiveQL
 - ☐ (b) Presto has better native integration with HBase and Cassandra
 - ☒ (c) **Presto executes queries entirely in memory without writing intermediate results to disk**
 - ☐ (d) Presto has native Python UDF support

Instructor Notes

Q3: Bucketing (a) divides *within* a partition by hash — it reduces join cost but does not skip data at the directory level. **Q4:** The key word is “intermediate results to disk.” Hive+MapReduce materialises intermediate data to HDFS between every MapReduce stage. Presto pipelines everything in memory, eliminating this I/O completely for most queries.

Instructions

State True or False and provide a **one-sentence justification** for full marks.

Statement	Answer
1. HiveQL supports full ACID transactions on all table types, just like MySQL.	False
2. Partitioning in Hive reduces query time by eliminating irrelevant data partitions from the HDFS scan.	True
3. Hive directly executes a HiveQL query on HDFS without any intermediate translation step.	False
4. Bucketing in Hive is particularly useful for optimising JOIN operations between two large tables.	True

Instructions

Answer each question in **100–150 words**. Include a diagram where indicated.

- D1.** Explain the **Hive architecture**: what is the role of HiveServer2, the Driver, and the Metastore? How is a HiveQL query compiled into executable jobs? Draw the component diagram.
- D2.** Differentiate between **partitioning** and **bucketing** in Hive. When would you choose each? Describe a scenario in Bangladesh e-commerce data where both are used together.
- D3.** What is “**schema-on-read**”? What are its advantages and disadvantages compared to “schema-on-write” in a traditional RDBMS such as MySQL? Give one example where schema-on-read causes a data quality problem.

Instructions

Answer in **200–300 words** with step-by-step reasoning and HiveQL code where applicable.

A1. Partitioning & Bucketing Design:

You have a 5 TB e-commerce transaction table with columns: (txn_id, user_id, product_id, amount, country, txn_date). Design a complete partitioning and bucketing strategy to optimise:

- Daily revenue reports filtered by country and txn_date
- JOINS between this table and a 10 M-row user profile table (keyed on user_id)

Write the complete CREATE TABLE DDL. Justify why you chose a specific number of buckets and which column(s) you partitioned on.

A2. Query Compilation to MapReduce:

Write HiveQL to find the **top 5 products by total revenue in 2023**. Then trace how Hive compiles this query into MapReduce stages: what is emitted by the Map phase, what is the shuffle key, and what does the Reduce phase compute? How would switching the execution engine from MapReduce to Tez change the execution plan?

Textbook & Reading References

Primary Textbooks for Week 7

- **[TW]** Tom White. *Hadoop: The Definitive Guide*, O'Reilly, 4th ed., 2015. **Ch. 12**
 - Ch. 12: Hive — comprehensive coverage of architecture, HiveQL, partitioning, bucketing, and UDFs
- **[SA]** Acharya & Chellappan. *Big Data and Analytics*, Wiley, 2015. **Ch. 5**
 - Ch. 5: Apache Hive; data warehousing on Hadoop
- **[LRU]** Leskovec, Rajaraman & Ullman. *Mining of Massive Datasets*, 3rd ed. **Ch. 2**
 - Ch. 2: SQL analogies in MapReduce; relational algebra in distributed systems

Seminal Papers (covered in Week 7, Part 2)

- Thusoo, A. et al. (2009). **Hive – A Warehousing Solution Over a Map-Reduce Framework**. *PVLDB 2009*.
- Thusoo, A. et al. (2010). **Hive – A Petabyte Scale Data Warehouse Using Hadoop**. *IEEE ICDE 2010*.
- Venkataraman, S. et al. (2019). **Presto: SQL on Everything**. *IEEE ICDE 2019*.

Questions & Discussion

Week 7, Part 1 | CSE 4345 Big Data Analytics

“The key insight behind Hive is not the SQL syntax — it is the Metastore: a shared, persistent catalogue that decouples the schema of your data from the files that hold it.”

— Adapted from Thusoo et al., ICDE 2010

Next Lecture: Week 7, Part 2

Seminal Papers: Thusoo et al. (2009, 2010) on Hive architecture and petabyte-scale warehousing

Case Study: **Spotify’s Hive-Based Analytics Platform** — 1 million tables, 600 billion events/day, artist royalty reporting at scale

Reading before Part 2: [TW] Ch. 12 · Thusoo et al. (2009) PVLDB paper (see references)